

# Clase 094 — Inyección NoSQL

Parte: 4 — Seguridad de aplicaciones web · Fuente: OWASP WSTG / Bug Bounty Bootcamp (Vickie Li)

 Duración estimada: 90 min · Nivel: Intermedio

## Objetivo

Entender que la inyección **no es exclusiva de SQL**: las bases NoSQL (MongoDB, etc.) tienen sus propios vectores. Aprenderás a explotar operadores de MongoDB y JavaScript del lado servidor para saltar autenticación y extraer datos.

 **Ética:** solo en laboratorios propios o autorizados. Estas técnicas modifican y exfiltran datos reales.

## Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** por qué el paso de objetos JSON habilita NoSQLi.
- 2. **Aplicar** operadores MongoDB ( `$ne` , `$gt` , `$regex` , `$where` ) como payloads.
- 3. **Saltar** autenticación con inyección de operadores.
- 4. **Extraer** datos con NoSQLi ciega basada en `$regex` .
- 5. **Recomendar** validación de tipos y sanitización como defensa.

## Temas

| # | Tema                                                  | Por qué importa                |
|---|-------------------------------------------------------|--------------------------------|
| 1 | Modelo de datos NoSQL                                 | Cambia la forma del ataque     |
| 2 | Operadores de consulta MongoDB                        | Los payloads son operadores    |
| 3 | Inyección vía JSON vs. query string                   | El formato altera el vector    |
| 4 | Auth bypass con <code>\$ne</code> / <code>\$gt</code> | Impacto directo                |
| 5 | Blind NoSQLi con <code>\$regex</code>                 | Extracción carácter a carácter |
| 6 | <code>\$where</code> y JS server-side                 | Ejecución de lógica arbitraria |
| 7 | Defensa: validar tipos                                | Cierre del fallo               |

## Definiciones y características

- **NoSQL injection:** manipular consultas de bases no relacionales insertando operadores u objetos. Característica: se aprovecha del tipado débil del input.

- **Operador `$ne`** : "not equal". Característica: `{"$ne": null}` casi siempre es verdadero, ideal para bypass.
- **Operador `$regex`** : coincidencia por expresión regular. Característica: permite inferir datos carácter a carácter (blind).
- **`$where`** : ejecuta JavaScript en el servidor Mongo. Característica: potente y peligroso; puede permitir DoS o extracción.
- **Inyección de objeto**: enviar `{"$ne": ""}` donde se espera un string. Característica: posible cuando el backend no valida tipos.
- **Type juggling**: confusión de tipos entre cliente y servidor. Característica: base de muchos bypass NoSQL.

## Herramientas y preparación

- **OWASP Juice Shop** (usa MongoDB-like en algunos retos) o un lab **DVNA/NodeGoat**.
- **Burp Suite** para editar cuerpos JSON.
- **NoSQLMap** (opcional, para automatizar).

```
# NodeGoat como lab NoSQL en Node/Mongo
git clone https://github.com/OWASP/NodeGoat && cd NodeGoat && docker compose up
```

## Laboratorio guiado

 Solo en tu laboratorio.

1. Localiza un login que reciba JSON ( `{"username": "x", "password": "y"}` ).
2. Con Burp, cambia el body a inyección de operador:

```
{"username": "admin", "password": {"$ne": ""}}
```

3. Observa si se produce el **bypass de autenticación**.
4. Prueba la variante por query string: `username[$ne]=&password[$ne]=`.
5. Para NoSQLi ciega, usa `$regex` para adivinar la contraseña:

```
{"username": "admin", "password": {"$regex": "^a"}}
```

6. Itera el prefijo ( `^a`, `^ab`, ...) según la respuesta de login para reconstruir el valor.
7. Si el backend usa `$where`, prueba una condición JS y evalúa el riesgo (sin causar DoS).

## Ejercicios

1. Diferencia el payload en JSON del payload en query string para el mismo bypass.
2. Reconstruye una contraseña de 8 caracteres con `$regex` blind.
3. Explica por qué `{"$gt": ""}` también funciona como bypass.
4. Escribe la validación en Node que impediría el ataque (comprobar `typeof === 'string'`).
5. Investiga qué hace `$where` y por qué está desaconsejado.
6. Compara conceptualmente SQLi y NoSQLi: similitudes y diferencias.

## Reto verificable

Consigue un **bypass de autenticación** en un lab NoSQL (NodeGoat/Juice Shop) y luego extrae parcialmente una credencial con NoSQLi ciega por `$regex`. **Criterio de aceptación:** demuestras el login sin conocer la contraseña y recuperas al menos los primeros caracteres del valor real mediante `$regex`, documentando los payloads.

## Errores comunes

| Síntoma / mensaje                  | Causa y cómo arreglar                                            |
|------------------------------------|------------------------------------------------------------------|
| El operador no se interpreta       | El backend recibe string, no objeto; ajusta Content-Type/formato |
| <code>\$where</code> bloqueado     | Deshabilitado en el servidor; usa otros operadores               |
| Regex demasiado lento              | Usa anclas <code>^</code> y búsqueda incremental                 |
| Bypass no funciona en query string | La app parsea distinto; prueba notación <code>param[\$ne]</code> |
| Falsos positivos                   | Confirma con dos operadores distintos                            |

## Preguntas frecuentes

 **¿Por qué el JSON facilita NoSQLi?** Porque un campo que debería ser string puede convertirse en un objeto con operadores si el servidor no valida el tipo.

 **¿MongoDB es inseguro por diseño?** No; el problema es el código que pasa input sin validar tipos ni sanitizar. Con validación estricta no hay NoSQLi.

 **¿Sirve sqlmap para NoSQL?** No. Para NoSQL existe NoSQLMap y payloads manuales; los conceptos son análogos pero la sintaxis difiere.

## Referencias

- OWASP Testing for NoSQL Injection (WSTG).
- PortSwigger NoSQL injection: <https://portswigger.net/web-security/nosql-injection>
- MongoDB Query Operators: <https://www.mongodb.com/docs/manual/reference/operator/query/>

## Siguiente clase

Clase 095 - Inyeccion de comandos del sistema operativo